

Lab 2: Asymmetric Key Cryptography (Advanced)

Exercises 1. Manual RSA Calculation: Given $p = 3$ and $q = 11$:

- Calculate $n = p * q$.
- Calculate $\phi(n) = (p-1) * (q-1)$.
- Choose $e = 3$. Find d such that $(e * d) \bmod \phi(n) = 1$.
- Encrypt the message $M = 5$ using $C = M^e \bmod n$.
- Decrypt C using $M = C^d \bmod n$.

2. Python RSA Encryption: Use the `cryptography` library to:

- Generate an RSA key pair.
- Encrypt the message "Top Secret" using the public key with OAEP padding.
- Decrypt it using the private key.

3. Diffie-Hellman Key Exchange Simulation (Making Things):

- **Problem:** Implement a Python script to simulate the Diffie-Hellman key exchange protocol between two parties (Alice and Bob) to establish a shared secret.

Solution

Exercise 1: Manual RSA Calculation

Given:

- $p=3$
- $q=11$
- $e=3$
- Message $M=5$

Step 1 :Calculate n

Formula:

$$n=p \times q$$

So:

$$n=3 \times 11=33$$

Therefore:

$$n = 33$$

Step 2: Calculate $\phi(n)$

Formula:

$$\phi(n) = (p-1)(q-1)$$

Therefore:

$$\phi(n) = (3-1)(11-1) = 2 \times 10 = 20$$

Therefore:

$$\phi(n) = 20$$

Step 3: Find d

We need:

$$(e \times d) \bmod \phi(n) = 1$$

We know:

$$e = 3$$

So:

$$(3d) \bmod 20 = 1$$

Try $d = 7$:

$$3 \times 7 = 21 \quad 21 \bmod 20 = 1$$

Therefore:

$$d = 7$$

Step 4: Encrypt $M=5$

Formula:

$$C = M \bmod n$$

Substitute:

$$C = 5 \bmod 33 \quad 5 \times 5 = 25$$

Now:

$$25 \bmod 33 = 25$$

Therefore:

$$\text{Ciphertext } C = 25$$

Step 5: Decrypt

Formula:

$$M = C \bmod n$$

Substitute:

$M=267 \bmod 33$

The result is:

$M=5$

Therefore, the original message is successfully recovered.

Final answers for Exercise 1

Parameter	Value
p	3
q	11
n	33
$\phi(n)$	20
e	3
d	7
Original message M	5
Encrypted C	26
Decrypted M	5

RSA flow:

5encrypt with $e=3$ 26decrypt with $d=7$ 5

Exercise 2:Python RSA Encryption

Commands used:

Checked if cryptography is installed

Command used:

```
python3 -c "import cryptography; print(cryptography.__version__)"
```

Created the Python file:

```
nano rsa_encryption.py
```

```
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes
```

```
# Generate RSA private key
```

```
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)
```

```
# Get the corresponding public key
```

```
public_key = private_key.public_key()
```

```
# Message to encrypt
```

```
message = b"Top Secret"
```

```
# Encrypt using the public key and OAEP padding
```

```
ciphertext = public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

```
# Decrypt using the private key
plaintext = private_key.decrypt(
    ciphertext,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)
```

```
# Display results
print("Original message:", message.decode())
print("Encrypted message:", ciphertext.hex())
print("Decrypted message:", plaintext.decode())
```

Ctrl + O:
Ctrl + X

Then:

```
python3 rsa_encryption.py
```



The image shows a Kali Linux virtual machine window with the nano text editor open. The editor contains a Python script for RSA encryption and decryption using the cryptography library. The script generates a 2048-bit RSA private key, extracts the public key, and encrypts the message "Top Secret" using OAEP padding with SHA256. It then decrypts the ciphertext back to the original message. The script uses color-coded syntax highlighting: red for comments, blue for imports, green for strings, and purple for None values.

```
GNU nano 9.1
from cryptography.hazmat.primitives.asymmetric import rsa, padding
from cryptography.hazmat.primitives import hashes

# Generate RSA private key
private_key = rsa.generate_private_key(
    public_exponent=65537,
    key_size=2048
)

# Get the corresponding public key
public_key = private_key.public_key()

# Message to encrypt
message = b"Top Secret"

# Encrypt using the public key and OAEP padding
ciphertext = public_key.encrypt(
    message,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Decrypt using the private key
plaintext = private_key.decrypt(
    ciphertext,
    padding.OAEP(
        mgf=padding.MGF1(algorithm=hashes.SHA256()),
        algorithm=hashes.SHA256(),
        label=None
    )
)

# Display results
print("Original message:", message.decode())
print("Encrypted message:", ciphertext.hex())
print("Decrypted message:", plaintext.decode())
```

At the bottom of the window, there is a menu bar with the following options: ^G Help, ^X Exit, ^O Write Out, ^R Read File, ^F Where Is, ^\ Replace, ^K Cut, ^U Paste, ^T Execute, ^J Justify, ^C Locate, and ^_ Go To.

```
kali@olivil: ~/INT306_Cryptography
$ python3 -c "import cryptography; print(cryptography.__version__)"
47.0.0

(kali@olivil)~/INT306_Cryptography
$ nano rsa_encryption.py

(kali@olivil)~/INT306_Cryptography
$ python3 rsa_encryption.py
Original message: Top Secret
Encrypted message: 5140e82349acc85b3aa5a81a62c7d4bcf35ac04ba7b2de13cd61bc7b62ee4102f8b7ede4857d94e4a1653d2926d9b67e9949adeda8022302713c29ff4276f3b75a08f7e07c0e3d7f8f44d2812994f0da2873fe4d9dba23642c9b54d735d1a7c02b9e126bc3634bbb593a86da446d0d9f426e025da0fd1df5d233ce3dbabfb8def371b63e318f9ebd0d5f8d3be9b0991a9e43f934b30c472303114d9f84c143120c6980a12f4c0fb6b948a3aea6bed7eb5f638aba9a9921995366250ee34648b045ad558f5da9a6e6ee0fe444955fb5ff4368ea16c85cbcb2b143c3b887bb51746f085d038712f1e6a83bd113bd4c843229905
Decrypted message: Top Secret

(kali@olivil)~/INT306_Cryptography
$ python3 rsa_encryption.py
Original message: Top Secret
Encrypted message: b0956009a76dfd98cfff39e5eb2157429951515b931bf715b8a4e61226b9b15da2463e4da42f0ec0957166c3cde70a616250ccd879cf660026014c2ee84641a3c8b87e96b39cce2108c0189d300e238c3445eae8c367397d28caaa922ea7f854fd6d19c712eeffd2c8eda1362792c9784d7a6aea02d05d0ef98a067267679536a1de197ffc6c4b19c4ecff6bcbf81697c13818a6bcc49274a73f6655eb0f2a37802f381fc4b653b34c2a259d4a07d7c5adfcacf024c571ae9378a30d6497ecdafa5b3df492d0003b81a3c969904cea04a19ad735d48618fc2f8c0b6a37ef4b52d42112fc1c7d9207af0cbf817b5c77731b473df
Decrypted message: Top Secret

(kali@olivil)~/INT306_Cryptography
$
```

The RSA encryption program successfully generated a 2048-bit RSA key pair and encrypted the message “**Top Secret**” using the public key with OAEP padding. The ciphertext was successfully decrypted using the private key, recovering the original message “**Top Secret**”, confirming that RSA encryption and decryption worked correctly.

Exercise 3: Diffie-Hellman Key Exchange Simulation

Commands used:

Create the Python file

```
nano diffie_hellman.py
```

Bash:

```
# Diffie-Hellman Key Exchange Simulation
```

```

# Public parameters
p = 23
g = 5

# Alice's private key
alice_private = 6

# Bob's private key
bob_private = 15

# Calculate public keys
alice_public = pow(g, alice_private, p)
bob_public = pow(g, bob_private, p)

# Calculate shared secrets
alice_shared_secret = pow(bob_public, alice_private, p)
bob_shared_secret = pow(alice_public, bob_private, p)

# Display results
print("Public prime (p):", p)
print("Public generator (g):", g)

print("\nAlice's private key:", alice_private)
print("Alice's public key:", alice_public)

print("\nBob's private key:", bob_private)
print("Bob's public key:", bob_public)

print("\nAlice's shared secret:", alice_shared_secret)
print("Bob's shared secret:", bob_shared_secret)

# Verify that both secrets are equal
if alice_shared_secret == bob_shared_secret:
    print("\nKey exchange successful!")
    print("Both Alice and Bob have the same shared secret.")
else:
    print("\nKey exchange failed!")

```

Saveed:

Ctrl + O y

Ctrl + X

Comand used:

python3 diffie_hellman.py

```
GNU nano 9.1
# Diffie-Hellman Key Exchange Simulation

# Public parameters
p = 23
g = 5

# Alice's private key
alice_private = 6

# Bob's private key
bob_private = 15

# Calculate public keys
alice_public = pow(g, alice_private, p)
bob_public = pow(g, bob_private, p)

# Calculate shared secrets
alice_shared_secret = pow(bob_public, alice_private, p)
bob_shared_secret = pow(alice_public, bob_private, p)

# Display results
print("Public prime (p):", p)
print("Public generator (g):", g)

print("\nAlice's private key:", alice_private)
print("Alice's public key:", alice_public)

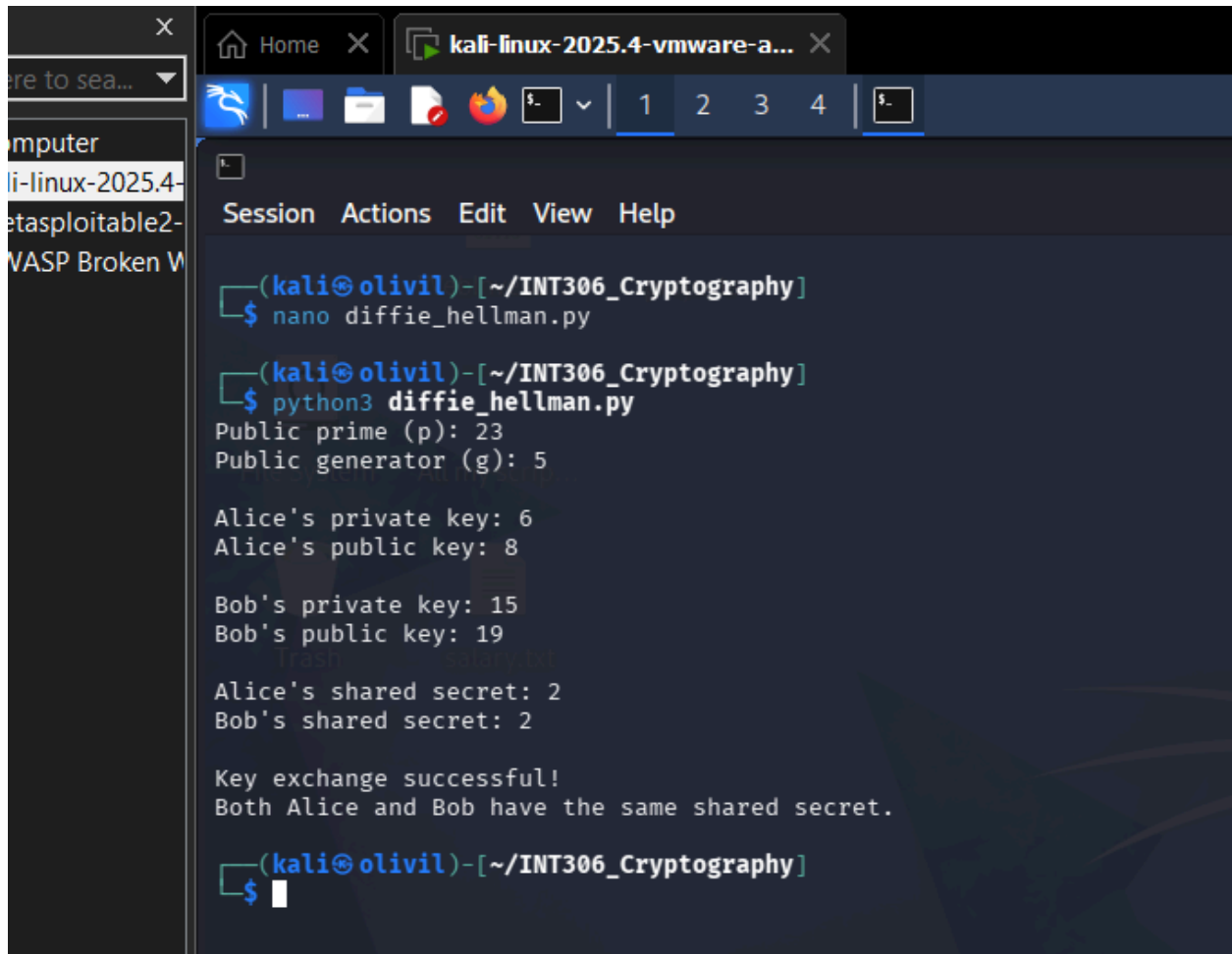
print("\nBob's private key:", bob_private)
print("Bob's public key:", bob_public)

print("\nAlice's shared secret:", alice_shared_secret)
print("Bob's shared secret:", bob_shared_secret)

# Verify that both secrets are equal
if alice_shared_secret == bob_shared_secret:
    print("\nKey exchange successful!")
    print("Both Alice and Bob have the same shared secret.")
else:
    print("\nKey exchange failed!")

^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify
^C Location  ^_ Go To L
```

puter, move the mouse pointer outside or press Ctrl+Alt.



```
(kali@olivil)-[~/INT306_Cryptography]
$ nano diffie_hellman.py

(kali@olivil)-[~/INT306_Cryptography]
$ python3 diffie_hellman.py
Public prime (p): 23
Public generator (g): 5

Alice's private key: 6
Alice's public key: 8

Bob's private key: 15
Bob's public key: 19

Alice's shared secret: 2
Bob's shared secret: 2

Key exchange successful!
Both Alice and Bob have the same shared secret.

(kali@olivil)-[~/INT306_Cryptography]
$
```

The Diffie-Hellman simulation successfully established a shared secret between Alice and Bob without transmitting their private keys.

Both parties independently calculated the same shared secret value, confirming that the key exchange was successful.

Level 5: Digital Signatures and PKI (Expert)

Exercises

1. Tamper Detection: Sign a file, then modify one character in the file. Try to verify the signature again. What is the result?

2. Self-Signed Certificate: Create a self-signed X.509 certificate for the domain `mysite.local` valid for 365 days.

3. Trust Chain Analysis: Use `openssl s\_client -connect google.com:443 -showcerts` to view the certificate chain. Identify the Root CA and the Intermediate CA.

Exercise 1: Tamper Detection

Commands used:

Generate the RSA private key:

```
openssl genrsa -out private.pem 2048
```

checked:

```
ls -l private.pem
```

Generate the public key:

```
openssl rsa -in private.pem -pubout -out public.pem
```

Then checked both:

```
ls -l private.pem public.pem
```

Created the message

```
echo "Hello World" > hello.txt
```

Checked it:

```
cat hello.txt
```

Signed the original file

```
openssl dgst -sha256 -sign private.pem -out signature.bin hello.txt
```

Checked the signature:

```
ls -lh signature.bin
```

Verified the original signature

```
openssl dgst -sha256 -verify public.pem -signature signature.bin hello.txt
```

Modify one character

```
sed -i 's/World/world/' hello.txt
```

Checked the modified file:

```
cat hello.txt
```

Verified the signature again

```
openssl dgst -sha256 -verify public.pem -signature signature.bin hello.txt
```

The screenshot shows a terminal window titled "kali-linux-2025.4-vmware-a...". The terminal prompt is "(kali@olivil)-[~/INT306_Cryptography]". The user enters the command "openssl genrsa -out private.pem 2048". The prompt changes to "(kali@olivil)-[~/INT306_Cryptography]" and the user enters "ls -l private.pem". The output shows the file "private.pem" with permissions "-rw-", size "1", owner "kali", group "kali", size "1708", and date "Aug 20 19:41". The prompt changes to "(kali@olivil)-[~/INT306_Cryptography]" and the user enters "openssl rsa -in private.pem -pubout -out public.pem". The output shows "writing RSA key". The prompt changes to "(kali@olivil)-[~/INT306_Cryptography]" and the user enters "ls -l private.pem public.pem". The output shows two files: "private.pem" with permissions "-rw-", size "1", owner "kali", group "kali", size "1708", and date "Aug 20 19:41"; and "public.pem" with permissions "-rw-rw-r--", size "1", owner "kali", group "kali", size "451", and date "Aug 20 19:42". The prompt changes to "(kali@olivil)-[~/INT306_Cryptography]" and the user enters a blank line.

```
(kali@olivil)-[~/INT306_Cryptography]
$ openssl genrsa -out private.pem 2048

(kali@olivil)-[~/INT306_Cryptography]
$ ls -l private.pem
-rw- 1 kali kali 1708 Aug 20 19:41 private.pem

(kali@olivil)-[~/INT306_Cryptography]
$ openssl rsa -in private.pem -pubout -out public.pem
writing RSA key

(kali@olivil)-[~/INT306_Cryptography]
$ ls -l private.pem public.pem
-rw- 1 kali kali 1708 Aug 20 19:41 private.pem
-rw-rw-r-- 1 kali kali 451 Aug 20 19:42 public.pem

(kali@olivil)-[~/INT306_Cryptography]
$
```

The screenshot shows a terminal window titled "kali-linux-2025.4-vmware-a...". The terminal prompt is "(kali@olivil)-[~/INT306_Cryptography]". The user enters the command "echo 'Hello World' > hello.txt". The prompt changes to "(kali@olivil)-[~/INT306_Cryptography]" and the user enters "cat hello.txt". The output shows "Hello World".

```
(kali@olivil)-[~/INT306_Cryptography]
$ echo "Hello World" > hello.txt

(kali@olivil)-[~/INT306_Cryptography]
$ cat hello.txt
Hello World
```

```
(kali@olivil)-[~/INT306_Cryptography]
$ openssl dgst -sha256 -sign private.pem -out signature.bin hello.txt

(kali@olivil)-[~/INT306_Cryptography]
$ ls -lh signature.bin
-rw-rw-r-- 1 kali kali 256 Aug 20 19:45 signature.bin

(kali@olivil)-[~/INT306_Cryptography]
$ openssl dgst -sha256 -verify public.pem -signature signature.bin hello.txt
Verified OK

(kali@olivil)-[~/INT306_Cryptography]
$ sed -i 's/World/world/' hello.txt

(kali@olivil)-[~/INT306_Cryptography]
$ cat hello.txt
Hello world

(kali@olivil)-[~/INT306_Cryptography]
$ openssl dgst -sha256 -verify public.pem -signature signature.bin hello.txt
Verification failure
802771AF297F0000:error:02000068:rsa routines:ossl_rsa_verify:bad signature:../crypto/rsa/rsa_sign.c:441:
802771AF297F0000:error:1C880004:Provider routines:rsa_verify_directly:RSA lib:providers/implementations/
signature/rsa_sig.c:1057:

(kali@olivil)-[~/INT306_Cryptography]
$
```

The original file produced a successful signature verification. After changing one character, verification failed, demonstrating that digital signatures can detect unauthorized modification of data.

Exercise 2: Self-Signed Certificate

Commands used:

Generated the certificate and private key:

```
openssl req -x509 -newkey rsa:4096 -keyout key.pem -out cert.pem -days 365 -nodes
-subj "/CN=mysite.local"
```

Checked that the files were created

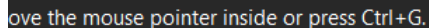
```
ls -lh key.pem cert.pem
```

Displayed the certificate

```
openssl x509 -in cert.pem -text -noout
```

Verified the self-signed certificate

```
openssl verify -CAfile cert.pem cert.pem
```



(kali@olivil)-[~/INT306_Cryptography]

\$ ls -lh key.pem cert.pem

```
-rw-rw-r-- 1 kali kali 1.8K Aug 20 20:15 cert.pem
-rw----- 1 kali kali 3.2K Aug 20 20:15 key.pem
```

(kali@olivil)-[~/INT306_Cryptography]

\$ openssl x509 -in cert.pem -text -noout

Certificate:

Data:

Version: 3 (0x2)

Serial Number:

09:ef:2a:81:be:97:c8:c9:0c:54:65:79:8a:21:ba:2a:bd:e7:f1:47

Signature Algorithm: sha256WithRSAEncryption

Issuer: CN=mysite.local

Validity

Not Before: Aug 21 00:15:36 2026 GMT

Not After : Aug 21 00:15:36 2027 GMT

Subject: CN=mysite.local

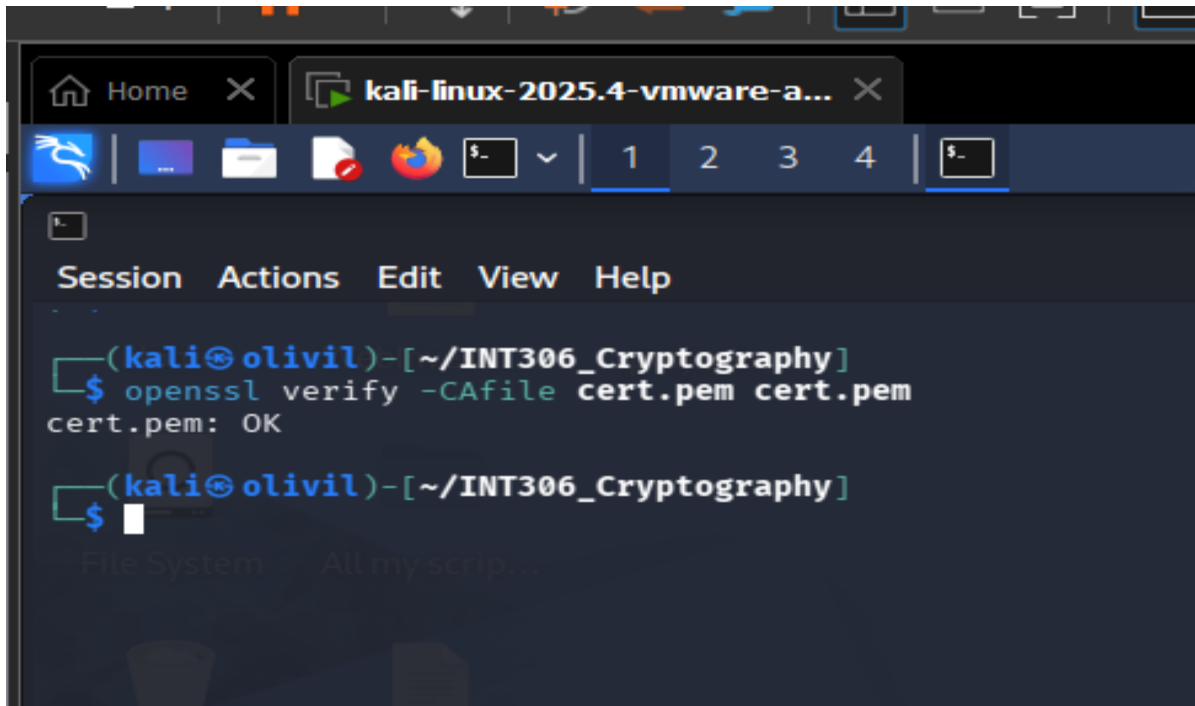
Subject Public Key Info:

Public Key Algorithm: rsaEncryption

Public-Key: (4096 bit)

Modulus:

```
00:be:c2:9a:32:5d:ca:30:e0:98:b1:63:6d:61:38:
5a:5e:02:32:c5:14:64:7c:cf:67:c0:3e:f0:88:2d:
a2:93:29:29:17:98:55:60:c5:98:29:c2:02:cf:f0:
bf:0a:4f:25:3a:c0:cd:fb:98:f4:60:53:cb:ee:1b:
6f:e8:af:3c:72:a3:f4:8d:9c:50:f4:b9:82:a9:32:
e5:f4:95:77:7a:a4:d2:8d:02:e9:08:13:4e:1d:bb:
3e:91:83:2d:2c:5f:59:4e:88:6b:ab:ce:95:d5:2a:
b8:2e:aa:bc:b8:0b:ae:37:8f:8f:b7:4b:1b:df:3a:
fc:f3:0d:9c:1e:23:d6:d8:e8:03:e8:bd:3d:c5:26:
2d:8b:84:eb:f4:34:51:a5:df:68:a8:b8:1b:e8:24:
38:d2:0f:5a:02:e3:ad:1b:f5:96:e8:1d:cc:b9:ea:
9e:8e:0f:0e:02:bd:08:98:8d:0f:79:f9:e3:2f:89:
ba:f4:1e:93:09:34:25:20:d6:b2:5f:5d:91:56:29:
58:59:12:87:dc:d9:c7:b8:17:41:17:e4:06:a2:0e:
5b:75:a3:16:39:39:70:2a:8c:db:c6:84:5a:1d:76:
33:1f:16:e5:8e:39:9b:bc:66:c5:5f:07:54:6d:be:
7f:89:21:73:66:da:f3:b4:91:7f:f6:7d:1d:26:51:
6d:ab:1d:ab:7b:ff:c9:ec:b6:50:65:f1:ef:d7:57:
10:98:d9:f9:a5:db:9c:73:24:cd:57:24:88:23:7f:
ff:cf:a9:29:cf:64:36:5d:51:f8:63:d8:ca:8d:b0:
c5:53:a5:72:01:6e:d6:f5:bd:17:2f:01:b6:77:3a:
50:a7:ea:df:0f:6c:c2:5f:49:5e:7c:60:a0:6d:ea:
b3:19:00:34:db:a4:18:c4:db:18:eb:0d:c6:ec:49:
f5:02:e4:10:72:66:18:7a:ed:f9:67:38:98:e4:b7:
45:2b:36:c6:72:f3:55:6d:5c:69:a6:85:95:eb:8b:
aa:11:9c:37:19:c1:eb:7c:ae:a8:81:bd:75:ff:4a:
```



A self-signed X.509 certificate was successfully created for `mysite.local` using a 4096-bit RSA key and a 365-day validity period. Because the certificate is self-signed, it signs itself rather than relying on an external Certificate Authority.

Exercise 3: Trust Chain Analysis

This exercise examines Google's HTTPS certificate chain.

Commands Used to connect to Google:

```
openssl s_client -connect google.com:443 -showcerts
```

displayed the certificate chain

```
openssl s_client -connect google.com:443 -showcerts </dev/null
```

Identify the certificates

```
openssl s_client -connect google.com:443 -showcerts </dev/null 2>/dev/null
```



```
-----BEGIN CERTIFICATE-----
MAKGHW0dHA6Ly9wa2kuZ29vZy9nc3IxL2dzcjEuY3J0MDIGA1UdHwQrMCKwJ6AL
oCOGIWh0dHA6Ly9jcmwucGtpLmdvb2cvZ3NyMS9nc3IxLmNybdA7BgNVHSAENDAy
MAgGBmeBDAECAITAIBgZngQwBAGIwDQYLKwYBBAHWeQIFAwIwDQYLKwYBBAHWeQIF
AwMwDQYJKoZIhvcNAQELBQADggEBADSkHrEoo9C0dhemMXoh6dFSPsjbdBZBiLg9
NR3t5P+T4Vxfq7vqfM/b5A3Ri1fyJm9bvhdGaJQ3b2t6yMAYN/olUazsaL+yyEn9
WprKAS0shIArAoyZl+tJaox118fessmXn1hIVw41oeQa1v1vg4Fv74zPl6/AhSrw
9U5pCZEt4Wi4wStz6dTZ/CLANx8LZh1J7QJVj2fhMtFTJr9w4z30Z209fOU0iOMy
+qduBmpvvYuR7hZL6Dupszfnw0Skfths18dG9ZKb59UhvmaSGZRVbNQpsg3BZlvi
d0LIK02d1xozclOzgJXPYovJJIultzkMu34qQb9Sz/yilrbCgj8=
-----END CERTIFICATE-----

Server certificate
subject=CN=*.google.com
issuer=C=US, O=Google Trust Services, CN=WR2

No client certificate CA names sent
Peer signing digest: SHA256
Peer signature type: ecdsa_secp256r1_sha256
Negotiated TLS1.3 group: X25519MLKEM768

SSL handshake has read 6212 bytes and written 1757 bytes
Verification: OK

New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
Protocol: TLSv1.3
Server public key is 256 bit
This TLS version forbids renegotiation.
Compression: NONE
Expansion: NONE
No ALPN negotiated
Early data was not sent
Verify return code: 0 (ok)

(kali@olivil)-[~/INT306_Cryptography]
```

The Google HTTPS certificate chain was examined using OpenSSL. The chain contains the server certificate and certificates issued by Certificate Authorities, with the intermediate CA linking the server certificate to a trusted root CA.

Level 6: Real-world Protocols and Advanced Topics (Expert)

TLS (Transport Layer Security)

Exercises

1. Cipher Suite Inspection: Use ``nmap --script ssl-enum-ciphers -p 443 google.com`` to list the supported cipher suites. Which ones are considered "weak"?

Exercise 1:Cipher Suite Inspection

The purpose is to use Nmap to inspect the TLS/SSL cipher suites supported by Google and identify suites that are considered weak.

Commands Used:

```
nmap --script ssl-enum-ciphers -p 443 google.com
```

Looked for weak ciphers:

- RC4 — weak/obsolete
- 3DES — considered weak/deprecated
- DES — weak
- NULL — provides no encryption
- EXPORT — intentionally weakened encryption
- anonymous (aNULL) — no authentication
- MD5 — obsolete for modern TLS use
- Very old TLS 1.0/1.1 support — deprecated

Saved the output:

```
nmap --script ssl-enum-ciphers -p 443 google.com -oN cipher_scan.txt
```

Then:

```
cat cipher_scan.txt
```

```
kali@kali:~$ nmap --script ssl-enum-ciphers -p 443 google.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-20 20:58 -0400
Nmap scan report for google.com (216.58.223.206)
Host is up (0.0045s latency).
Other addresses for google.com (not scanned): 2c0f:fb50:4003:801::200e
rDNS record for 216.58.223.206: los02s03-in-f14.1e100.net

PORT      STATE SERVICE
443/tcp   open  https
| ssl-enum-ciphers:
|   TLSv1.0:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
|     compressors:
|       NULL
|     cipher preference: server
|     warnings:
|       64-bit block cipher 3DES vulnerable to SWEET32 attack
|   TLSv1.1:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
|     compressors:
|       NULL
|     cipher preference: server
|     warnings:
|       64-bit block cipher 3DES vulnerable to SWEET32 attack
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
```

(kali@olivil)-[~/INT306_Cryptography]

\$ nmap --script ssl-enum-ciphers -p 443 google.com -oN cipher_scan.txt

Starting Nmap 7.99 (https://nmap.org) at 2026-08-20 21:00 -0400

Nmap scan report for google.com (216.58.223.206)

Host is up (0.0069s latency).

Other addresses for google.com (not scanned): 2c0f:fb50:4003:803::200e

rDNS record for 216.58.223.206: los02s03-in-f14.1e100.net

PORT STATE SERVICE

443/tcp open https

| ssl-enum-ciphers:

| TLSv1.0:

| ciphers:

| TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A

| TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A

| TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A

| TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C

| compressors:

| NULL

| cipher preference: server

| warnings:

| 64-bit block cipher 3DES vulnerable to SWEET32 attack

| TLSv1.1:

| ciphers:

| TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A

| TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A

| TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A

| TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C

| compressors:

| NULL

| cipher preference: server

| warnings:

| 64-bit block cipher 3DES vulnerable to SWEET32 attack

| TLSv1.2:

| ciphers:

| TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A

| TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A

| TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A

| TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A

| TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A

```

compressors:
  NULL
cipher preference: client
warnings:
  64-bit block cipher 3DES vulnerable to SWEET32 attack
TLSv1.3:
  ciphers:
    TLS_AKE_WITH_AES_128_GCM_SHA256 (X25519MLKEM768) - A
    TLS_AKE_WITH_AES_256_GCM_SHA384 (X25519MLKEM768) - A
    TLS_AKE_WITH_CHACHA20_POLY1305_SHA256 (X25519MLKEM768) - A
  cipher preference: client
_ least strength: C

Nmap done: 1 IP address (1 host up) scanned in 6.63 seconds

(kali@olivil)-[~/INT306_Cryptography]
$ cat cipher_scan.txt
# Nmap 7.99 scan initiated Thu Aug 20 21:00:08 2026 as: /usr/lib/nmap/nmap --
enum-ciphers -p 443 -oN cipher_scan.txt google.com
Nmap scan report for google.com (216.58.223.206)
Host is up (0.0069s latency).
Other addresses for google.com (not scanned): 2c0f:fb50:4003:803::200e
rDNS record for 216.58.223.206: los02s03-in-f14.1e100.net

PORT      STATE SERVICE
443/tcp   open  https
ssl-enum-ciphers:
  TLSv1.0:
    ciphers:
      TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
      TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
      TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
      TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
    compressors:
      NULL
    cipher preference: server
    warnings:
      64-bit block cipher 3DES vulnerable to SWEET32 attack
  TLSv1.1:
    ciphers:
      TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
      TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A

```

Nmap identified TLS_RSA_WITH_3DES_EDE_CBC_SHA as the weakest cipher suite, receiving a C rating because 3DES is vulnerable to the SWEET32 attack. The scan also showed modern TLS 1.2 and TLS 1.3 cipher suites using AES-GCM and ChaCha20-Poly1305, which provide stronger security.

Exercise 2:GPG File Encryption

Commands Used:

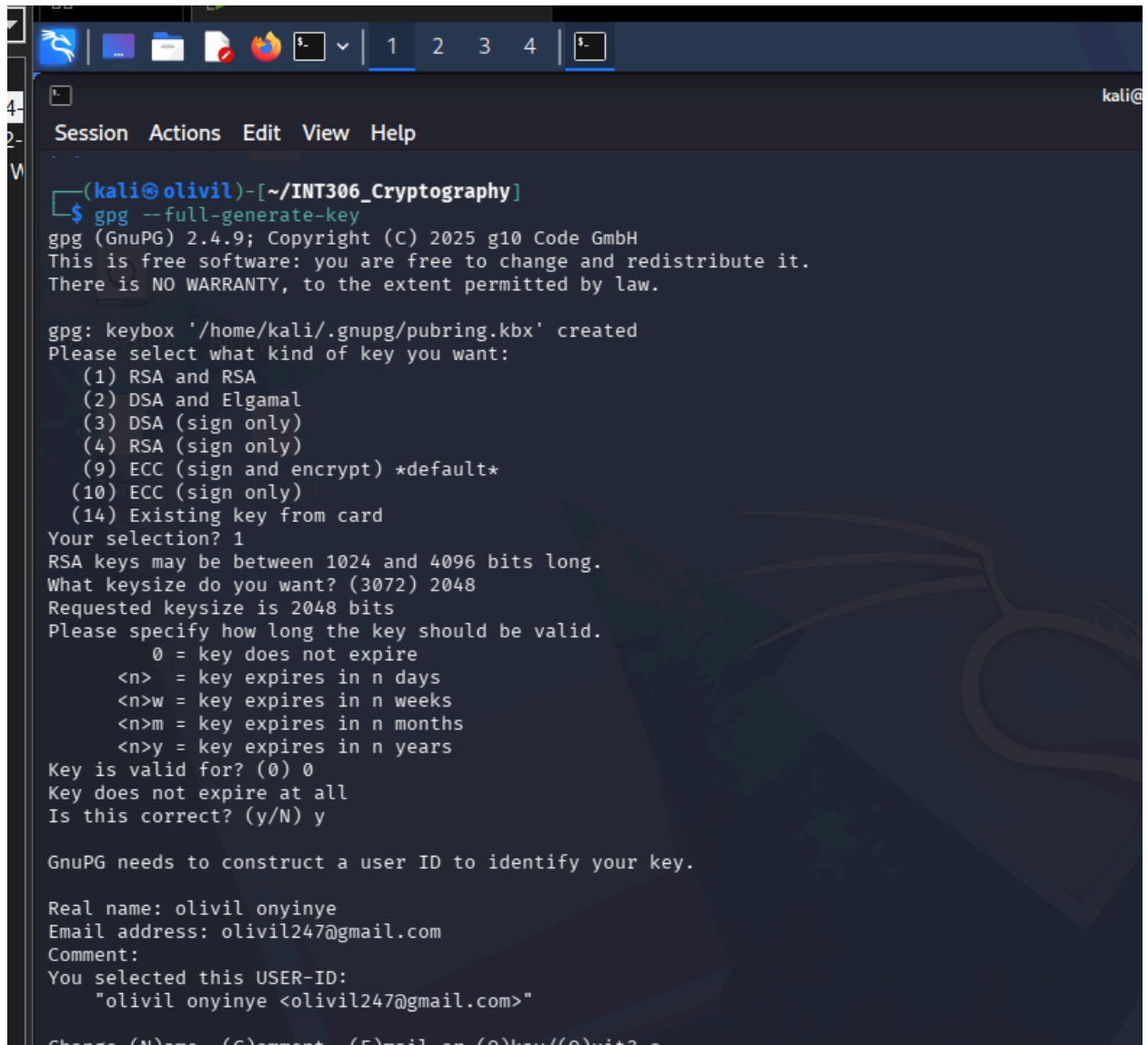
Generate GPG key pair

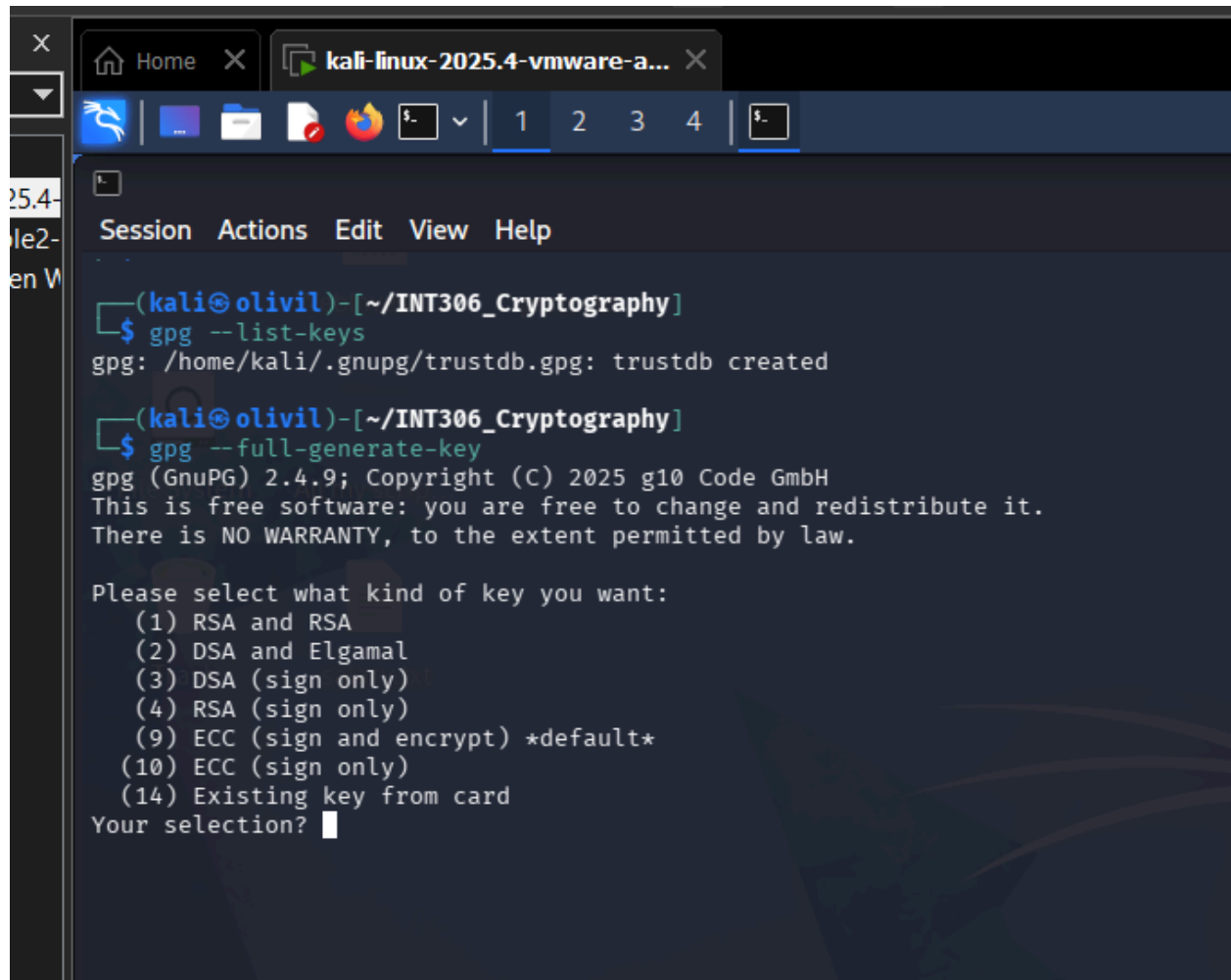
```
gpg --full-generate-key
```

Verified that the key was created

```
gpg --list-keys
```

```
gpg --full-generate-key
```



A screenshot of a Kali Linux terminal window. The window title is 'kali-linux-2025.4-vmware-a...'. The terminal shows the user 'olivil' at the prompt '(kali@olivil)-[~/INT306_Cryptography]'. The first command is 'gpg --list-keys', which outputs 'gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created'. The second command is 'gpg --full-generate-key', which outputs 'gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH' and 'This is free software: you are free to change and redistribute it. There is NO WARRANTY, to the extent permitted by law.' It then prompts 'Please select what kind of key you want:' followed by a list of options: (1) RSA and RSA, (2) DSA and Elgamal, (3) DSA (sign only), (4) RSA (sign only), (9) ECC (sign and encrypt) *default*, (10) ECC (sign only), and (14) Existing key from card. The prompt 'Your selection?' is followed by a cursor.

```
(kali@olivil)-[~/INT306_Cryptography]
$ gpg --list-keys
gpg: /home/kali/.gnupg/trustdb.gpg: trustdb created

(kali@olivil)-[~/INT306_Cryptography]
$ gpg --full-generate-key
gpg (GnuPG) 2.4.9; Copyright (C) 2025 g10 Code GmbH
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.

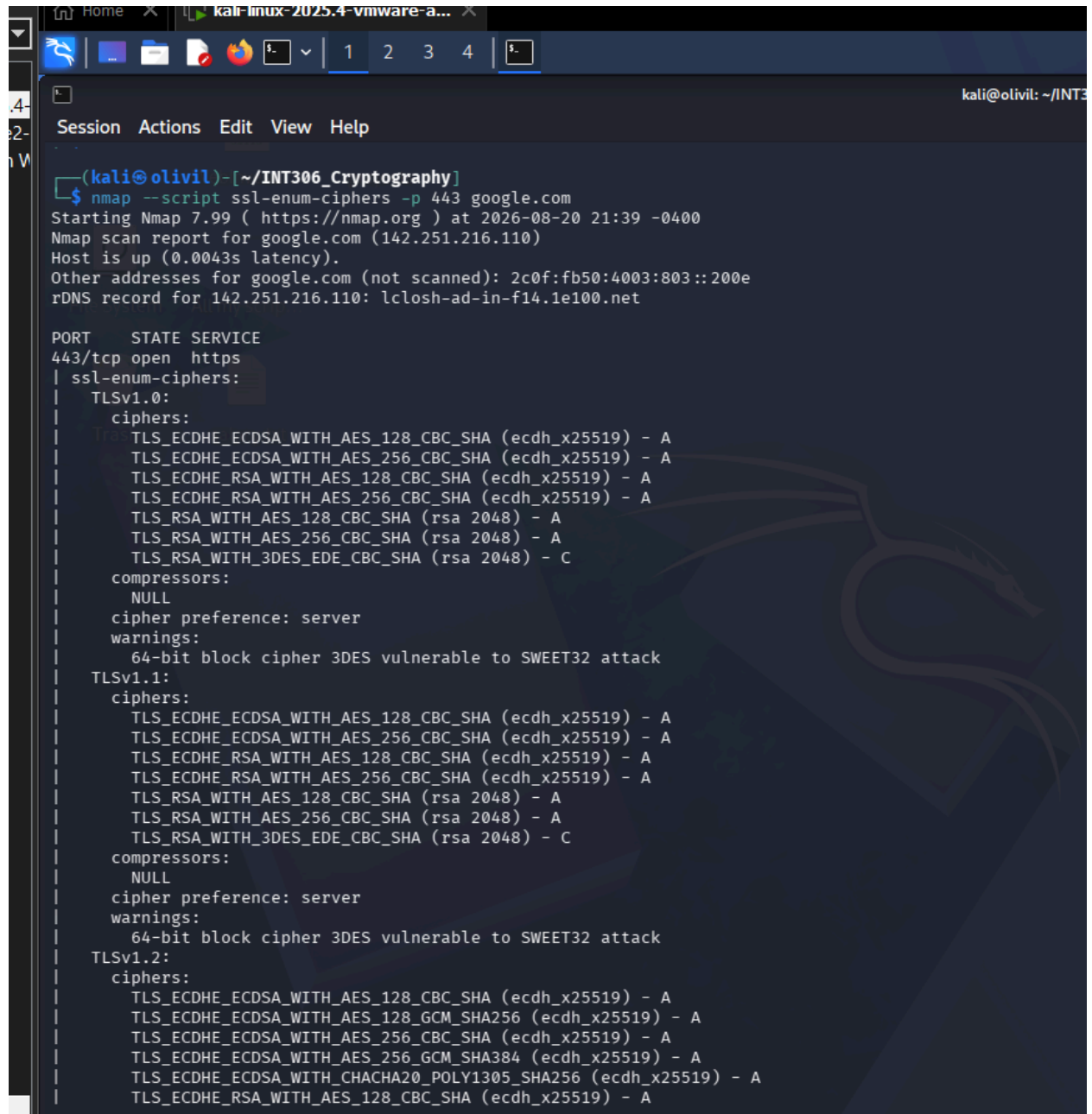
Please select what kind of key you want:
  (1) RSA and RSA
  (2) DSA and Elgamal
  (3) DSA (sign only)
  (4) RSA (sign only)
  (9) ECC (sign and encrypt) *default*
 (10) ECC (sign only)
 (14) Existing key from card
Your selection? █
```

A GPG RSA key pair was successfully generated using GnuPG. A file was encrypted for the generated recipient and successfully decrypted, demonstrating secure public-key encryption and recovery of the original plaintext using the corresponding private key.

Exercise 1: Cipher Suite Inspection

Commands Used:

```
nmap --script ssl-enum-ciphers -p 443 google.com
```



```
(kali@olivil)-[~/INT306_Cryptography]
$ nmap --script ssl-enum-ciphers -p 443 google.com
Starting Nmap 7.99 ( https://nmap.org ) at 2026-08-20 21:39 -0400
Nmap scan report for google.com (142.251.216.110)
Host is up (0.0043s latency).
Other addresses for google.com (not scanned): 2c0f:fb50:4003:803::200e
rDNS record for 142.251.216.110: lclosh-ad-in-f14.1e100.net

PORT      STATE SERVICE
443/tcp   open  https
| ssl-enum-ciphers:
|   TLSv1.0:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
|     compressors:
|       NULL
|     cipher preference: server
|     warnings:
|       64-bit block cipher 3DES vulnerable to SWEET32 attack
|   TLSv1.1:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
|       TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
|     compressors:
|       NULL
|     cipher preference: server
|     warnings:
|       64-bit block cipher 3DES vulnerable to SWEET32 attack
|   TLSv1.2:
|     ciphers:
|       TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
|       TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
|       TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
```

```

    TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
    compressors:
      NULL
    cipher preference: server
    warnings:
      64-bit block cipher 3DES vulnerable to SWEET32 attack
  TLSv1.2:
    ciphers:
      TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
      TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_ECDSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
      TLS_ECDHE_ECDSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_AES_256_GCM_SHA384 (ecdh_x25519) - A
      TLS_ECDHE_RSA_WITH_CHACHA20_POLY1305_SHA256 (ecdh_x25519) - A
      TLS_RSA_WITH_3DES_EDE_CBC_SHA (rsa 2048) - C
      TLS_RSA_WITH_AES_128_CBC_SHA (rsa 2048) - A
      TLS_RSA_WITH_AES_128_GCM_SHA256 (rsa 2048) - A
      TLS_RSA_WITH_AES_256_CBC_SHA (rsa 2048) - A
      TLS_RSA_WITH_AES_256_GCM_SHA384 (rsa 2048) - A
    compressors:
      NULL
    cipher preference: client
    warnings:
      64-bit block cipher 3DES vulnerable to SWEET32 attack
  TLSv1.3:
    ciphers:
      TLS_AKE_WITH_AES_128_GCM_SHA256 (X25519MLKEM768) - A
      TLS_AKE_WITH_AES_256_GCM_SHA384 (X25519MLKEM768) - A
      TLS_AKE_WITH_CHACHA20_POLY1305_SHA256 (X25519MLKEM768) - A
    cipher preference: client
    least strength: C

Nmap done: 1 IP address (1 host up) scanned in 4.92 seconds
-(kali@olivil)-[~/INT306_Cryptography]

```

Nmap identified several supported TLS cipher suites, with most receiving an A security rating. The TLS_RSA_WITH_3DES_EDE_CBC_SHA cipher was rated C because 3DES uses a 64-bit block size and is vulnerable to the SWEET32 attack. TLS 1.3 was also supported with modern AES-GCM and ChaCha20-Poly1305 encryption.

